

简单流水线 CPU 设计

通过前一章的学习实践，我们设计并实现了一个支持 20 条指令的单周期 CPU。本章我们将学习如何将单周期 CPU 改造成单发射五级流水线 CPU。从单周期 CPU 到流水线 CPU，对初学者来说还是有些难度的。为了让学习曲线不至于陡峭，我们进一步将简单流水线 CPU 的设计工作分解成三个逐级递进的小任务。

- **第一阶段**：我们将尝试在单周期 CPU 的基础上引入流水线，但是暂时不考虑处理各类相关所引发的冲突。
- **第二阶段**：我们会介绍流水线中各类相关所引发的冲突，并给出用阻塞方式解决冲突的设计方案。在本章的实践任务中，读者可根据我们给出的设计方案，基于第一阶段的 CPU 实现进行改进。
- **第三阶段**：我们会介绍流水线前递技术，并给出一种设计方案。在本章的实践任务中，读者可根据我们给出的设计方案，基于第二阶段的 CPU 实现进行设计和调整。

【本章学习目标】

- 继续建立设计方案与 Verilog 代码实现之间的认知联系。
- 形成良好的 Verilog 代码编写习惯。
- 进一步提升 CPU 功能仿真验证中调试错误的能力。

【本章实践目标】

本章有三个实践任务（见 5.5 节）。读者可以在学习本章内容的基础上完成这些任务，两者之间的对应关系如下：

- 5.1 节的内容对应实践任务 7（5.5.1 节）。
- 5.2 节的内容对应实践任务 8（5.5.2 节）。

□ 5.3 节的内容对应实践任务 9（5.5.3 节）。

5.1 不考虑相关冲突的流水线 CPU 设计

我们进入流水线 CPU 设计的阶段。这一节我们只解决两个设计问题：一是把单周期 CPU 的数据通路切分成多个阶段，二是让多个阶段“流起水”来。所谓“流起水”，就是在理想情况下，流水线每一级都有指令且每个时钟周期都能处理完一条指令。

5.1.1 添加流水级间缓存

回想一下前面第 3 章提到的流水线电路的一般性设计，我们知道将电路流水线化的初衷是缩短时序器件之间组合逻辑关键路径的时延，在不降低电路处理吞吐率的情况下提升电路的时钟频率。从电路设计最终的实现形式来看，是将一段组合逻辑按照功能划分为若干阶段，在各功能阶段的组合逻辑之间插入时序器件（通常是触发器），前一阶段的组合逻辑输出接入时序器件的输入，后一阶段的组合逻辑输入来自这些时序器件的输出。

5.1.1.1 流水线的划分

由于我们设计的 CPU 是一个数字逻辑电路，因此 CPU 电路的流水化也遵循上面的设计思路。在真实的 CPU 设计过程中，最困难的地方是决定将单周期 CPU 中的组合逻辑划分为多少个阶段以及各个阶段包含哪些功能。这个设计决策需要结合 CPU 产品的性能（含主频）、功耗、面积指标以及具体采用的工艺特性来完成。对初学者而言，这部分设计涉及的内容过多、过细、过深，因此我们将直接采用经典的单发射五级流水线划分。所划分的五级流水线从前往后依次为：**取指**（IF）阶段、**译码**（ID）阶段、**执行**（EXE）阶段、**访存**（MEM）阶段和**写回**（WB）阶段。取指阶段的主要功能是将指令取回，译码阶段的主要功能是解析指令生成控制信号并读取通用寄存器堆生成源操作数，执行阶段的主要功能是对源操作数进行算术逻辑类指令的运算或者访存指令的地址计算，访存阶段的主要功能是取回访存的结果，写回阶段的主要功能是将结果写入通用寄存器堆。结合这个流水线阶段的划分方案，我们将之前设计的单发射 CPU 的数据通路拆分为五段，并在各段之间加入触发器作为流水线缓存。

这里要解释下各级流水线间缓存数据所用的触发器如何命名。很多教科书都是以两级流水线的名字来标识这些触发器，例如，位于取指阶段和译码阶段之间的触发器

记作“IF/IDreg”。为了编码时命名简洁，本书中通常将触发器归到它的输出对应的那一个流水线阶段。因此，位于取指阶段和译码阶段之间的触发器记作“IDreg”。这种命名风格来自我们进行仿真波形调试时的观察习惯，即除了触发器时钟采样边沿附近那段建立保持时间（当采用零延迟仿真时，这段时间可以看作瞬时、无穷小）外，触发器中存储的内容都是与它输出那一级流水线的组合逻辑相关联的。这里所说的只是一种命名的习惯，没有对错、优劣之分。本书后面将统一采用这种单级流水线标识的方式。

5.1.1.2 流水线缓存中存放的内容

流水线缓存的内容分为控制内容和数据内容。

目前我们设计的流水线 CPU 只需要考虑各级流水线缓存是否有效，所以缓存中的控制内容只需要 1 位缓存有效位（valid）就可以了。其值为 0 表示这一级流水线缓存中存放的数据内容无效，其值为 1 表示存放的数据内容有效。

接下来的主要设计工作是考虑每一级缓存中有哪些数据内容。显然，原有单周期 CPU 中的信号线（**既包括数据信号也包括控制信号**）如果被插入了流水线缓存，那么**这些**流水线缓存的数据内容肯定要包含将被隔断的信号。我们加粗着重标识“这些”二字，是为了提醒各位一根信号线可能会被多级流水线缓存多次隔断。那么，这几级缓存的数据部分都要包含这根信号线所携带的数据信息。下面举三个例子来说明一下。

【例一】 指令在译码阶段得到其对通用寄存器的写使能信号和写地址信号，但是这些信号直到指令位于 WB 阶段才被使用，即这些信号被 EXE、MEM、WB 三级流水线缓存隔断，所以这些信号在这三级流水线的缓存中都要存放。

【例二】 指令在译码阶段生成 ALU 的控制信号 `alu_op`，在 EXE 阶段使用，中间被 EXE 级流水线缓存隔断，所以 EXE 级流水线缓存中需要存放 `alu_op`。

【例三】 ALU 的结果 `alu_res` 在 EXE 阶段产生，数据 RAM 读出的 load 访问结果在 MEM 阶段产生，那么从 ALU 结果和 load 访问结果中选出通用寄存器堆写入值的逻辑最早只能是在 MEM 阶段完成，所以 MEM 级流水线缓存中需要存放 `alu_res`。

总结一下，**插入流水线缓存后，从信号产生的流水线阶段开始到使用它的流水线阶段，途经的各级流水线缓存中都要存放该信号。**

5.1.2 同步读 RAM 的引入

前面在讲述单周期 CPU 数据通路设计时曾说过，为了保证单周期能完成一条指令，

暂时使用“异步读 RAM”来实现指令 RAM 和数据 RAM，等到设计流水线 CPU 的时候再调整回“同步读 RAM”。我们现在就来讨论这个设计调整。

首先我们回顾一下同步读 RAM 与异步读 RAM 在时序特性上的不同之处——同步读 RAM 一次读数操作需要跨越两个时钟周期，第一个时钟周期向 RAM 发出读使能和读地址，第二个时钟周期 RAM 才能返回读结果。

在使用异步读 RAM 的时候，CPU 在取指阶段向指令 RAM 发出读使能和读地址，同一时钟周期内就能得到指令 RAM 的返回结果（指令码），因此可以在时钟上升沿到来时将指令码写入译码阶段的流水线缓存中。显然，当我们把指令 RAM 从异步读 RAM 换为同步读 RAM 后，时钟上升沿到来时还无法得到指令码，所以需要调整 CPU 的数据通路设计。

一种设计考虑是，将取指阶段设计为占用两个时钟周期，第一个周期发送读使能和读地址，第二个周期获得 RAM 返回数据后再进入译码阶段。这显然是个笨办法。最理想的情况下，这个 CPU 也只能做到每两个周期处理一条指令，性能损失非常严重。

另一种设计考虑是，流水线缓存的触发器都采用时钟上升沿触发，而指令 RAM 采用时钟下降沿触发。这种设计属于“听上去不错”的解决方案。感兴趣的读者可以在 FPGA 上实现一下，就会发现最终实现的电路的频率会大幅度下降。其原因与 FPGA 的底层电路实现机制有关，此处就不深入解释了。其实不仅是 FPGA，即便是 ASIC 实现的时候，我们也要尽量避免在同一个设计中使用同一个时钟的上升沿和下降沿，以减轻物理设计的负担。

在否定了上面两种设计考虑后，我们考虑将指令 RAM 的访问分布到流水线中连续的两个阶段，这样既能满足同步读 RAM 的时序特性，也能达到每周期处理一条指令的吞吐率理论峰值。在不增加流水级数的情况下，只有两种设计方案：

- **方案一**：将指令 RAM 的读请求发起放在取指阶段，这样指令 RAM 的输出是在指令位于译码阶段时完成的。
- **方案二**：在“更新 PC 的阶段”发起指令 RAM 的读请求（也就是以 nextPC 为指令 RAM 的读地址，而不是以 PC 为指令 RAM 的读地址），这样指令 RAM 的输出是在指令位于取指阶段时完成的。

在方案二中，所谓“更新 PC 的阶段”并不是一个真正的流水线阶段，可以说它是一个“伪流水线阶段”，因为这个阶段自身没有流水线缓存，它的组合逻辑的输入来自 IF、ID 等阶段的流水线缓存，输出为 nextPC，我们记为 pre-IF 阶段。这一设计可以理解为将取指阶段拆分为 pre-IF 和 IF 两个阶段，pre-IF 阶段只负责生成 nextPC，nextPC

被送到 IF 阶段更新 PC 寄存器。习惯上, 我们还是将 pre-IF 和 IF 阶段合称为取指阶段。

那么这两种方案又该如何选择呢? 如果是采用 ASIC 方式实现, 第二种方案更好。因为对于 ASIC 实现来说, 同步读 RAM 的 clk-to-Q 延迟比它的输入端口的 Setup 延迟大, 也会远大于触发器的 clk-to-Q 延迟。如果采用第一种方案, 译码阶段就会出现“RAM 读出 → 通用寄存器堆读出”和“RAM 读出 → 指令译码 → ALU 源操作数选择”这两条路径, 两条路径上的通用寄存器读出和指令译码的时延已经不短, 再加上 RAM 读出所需的 RAM 的 clk-to-Q 延迟, 会导致整条路径的延迟变得非常大。如果采用 FPGA 来实现, 情况就有所不同了。由于 FPGA 上同步时序行为的 RAM 是用所谓的“block RAM”来实现的, 其对应的底层电路实现与前面提到的 ASIC 设计中的 RAM 是相同的。对于 Xilinx7 系列及更先进的 FPGA 来说, 这些 RAM 的最高频率通常在数百兆赫兹, 我们用 FPGA 实现一个像 CPU 这样略有些复杂的电路时, RAM 以外的逻辑部分至多达到 200MHz, 此时 block RAM 自身的 clk-to-Q 延迟就不再是主要矛盾了。如果说所使用的 RAM 容量不是特别大, 那么 FPGA 使用的 block RAM 数目并不会很多, 其引入的走线距离也不是很远, 那么 RAM 读出这一过程的总延迟也并不大。考虑到这种 FPGA 上的特殊情况, 第一种方案也有一些合理性。

本书中我们推荐采用第二种设计方案。虽然第一种设计方案在当前设计要求下比较容易实现, 但是在后续的实践中会带来一定的麻烦, 因为译码阶段的指令直接来自 RAM 的读出端口, 当译码阶段的指令因为阻塞需要持续多拍时, 如何维护 RAM 读出的指令不变是一个令人头疼的问题。如果读者愿意在实践中尝试第一种设计方案, 我们也不反对, 但是前提是对同步读 RAM 的时序行为已经了然于胸。我们还要特别提醒的是, 如果采用第一种方案, 一定不要把指令 RAM 读出的内容保存到译码阶段的流水线缓存中再去实现译码、读寄存器堆, 必须直接用指令 RAM 读出的内容去实现译码、读寄存器堆。

讨论完指令 RAM 的同步读 RAM 设计调整之后, 数据 RAM 的设计调整完全可以采用同样的思路来完成。提示一下, 数据 RAM 的读请求或写请求是在指令位于 EXE 阶段时发出的, 数据 RAM 的读数据是在指令位于 MEM 阶段时返回的。

同步读 RAM 的读保持功能

这里要讨论的“同步读 RAM 的读保持功能”是指对于有的 RAM 来说, 从采样到上一个有效的读命令的时钟沿开始, RAM 的 Q 端将保持这个读操作对应的数据, 直至采样到下一个有效的命令。请注意, 这里的命令必须是有效的, 例如, 读命令只有在 RAM 的片选使能信号有效的情况下才可能有效, 所以当 RAM 的片选使能信号无效时,

即使 RAM 的地址不停地发生变化, RAM 也没有接收到任何新的读命令。

对不同 IP 厂商提供的 RAM 来说, 这里所说的 RAM 读保持功能在细节上可能存在差异。譬如, 有的情况下 RAM 的 Q 端保持是在相邻的两个读命令之间实现的, 也就是说, 如果先发送一个读命令, 过一会儿再发送一个写命令的话, RAM 的 Q 端还是保持前一个读的结果。又譬如, 有的情况下要想实现 RAM 的 Q 端保持, 在发送完读命令之后, RAM 的时钟必须一直提供不能关断功能。虽然目前各 IP 提供厂商提供的 RAM 大多都具有读保持功能, 但如果要使用这个特性, 务必在使用前查阅 RAM 的技术文档, 明确相关行为。

虽然在本节所涉及的范围内不会用到 RAM 读保持功能, 但是, 在后续的实践中, 流水线 CPU 是可以被阻塞停顿的。这就意味着, 指令有可能在这一拍发出了 RAM 的读请求, 但是在下一拍无法进入后一个流水阶段, 此时就需要保证指令最终到达后一个流水阶段的时候, 仍然能够从 RAM 的输出端口上得到需要的正确内容。如果利用 RAM 的读保持功能, 可以适度简化 RAM 地址输入的生成逻辑, 只是 RAM 的片选使能信号需要控制得更精细一些。

5.1.3 调整更新 PC 的数据通路

因为计算跳转目标地址的信息来自指令码和通用寄存器堆, 所以转移指令最早在译码阶段就能够计算出正确的跳转方向和目标。为了尽可能减少控制相关带来的流水线阻塞, 我们将跳转的转移指令修改 PC 的时机安排在其处于译码阶段。此时应该注意, pre-IF 阶段的 nextPC 生成逻辑中, 来自 PC 相对转移指令跳转目标的那一支, 其跳转目标计算所用的 PC 是处在译码阶段的转移指令的 PC, 不是此时取指阶段的 PC。

5.1.4 不考虑相关冲突情况下流水线控制信号的设计

前面我们给出了流水线电路的一般性设计方法。这里我们结合不考虑相关冲突的流水线的实际情况, 对电路中与流水线相关的控制信号予以调整, 主要涉及各级 ready_go 信号如何设置。

- 对于 IF 阶段来说, 由于目前只从指令 RAM 中取回指令, 因此当指令位于取指阶段的时候, 指令 RAM 一定可以返回指令码, 于是取指阶段的 ready_go 信号恒为 1。
- 对于 ID 流水阶段来说, 由于译码、读寄存器堆都是一拍之内一定可以完成的, 所以译码阶段的 ready_go 信号恒为 1。

- 对于 EXE 阶段来说，由于目前处理的所有指令在这一阶段均只需要一拍就可以完成，所以 EXE 阶段的 `ready_go` 信号恒为 1。
- 对于 MEM 阶段来说，由于目前只从数据 RAM 中取回数据，因此当 load 类指令位于 MEM 阶段的时候，数据 RAM 一定可以返回数据，于是 MEM 阶段的 `ready_go` 信号恒为 1。
- 对于 WB 阶段来说，由于写回寄存器堆在一拍之内一定可以完成，因此 WB 阶段的 `ready_go` 信号恒为 1。

综上所述，五个阶段的 `ready_go` 信号都是 1。这里特意逐个说明一遍的目的有两个：一是提醒大家对这些信号为什么恒置 1 要心中有数；二是提醒大家不要急着把这些常值 1 带到生成其他控制信号的逻辑中进行化简。随着设计复杂度的提升，这些 `ready_go` 信号将变得越来越复杂。

至此，在不考虑相关冲突的情况下，一个支持 20 条 LoongArch 指令的流水线 CPU 就设计完成了。这个 CPU 虽然具备了流水处理的特点，但是它上面还只能运行特殊处理后的 LoongArch 可执行程序。

5.1.5 复位的处理

这里讨论教科书中较少涉及的一个话题——复位。既然是复位信号，那么在其有效期间就应该将 CPU 中所有软件可感知的状态都初始化到一个唯一确定的状态。这句话听起来有些绕，但实际上复位要达到的效果就是，无论一个 CPU 重启多少次，它每次运行同一个程序的行为都是一致的。例如，CPU 上电复位之后第一条指令从哪里取，这个状态就必须确定，指令手册在 6.3 节对此进行了明确规定。那么通用寄存器堆的状态呢？内存的状态呢？我们发现指令手册中对这些问题没有明确规定，这就意味着 CPU 的电路不用在复位信号有效期间将这些存储中的值置为某个确定的值，它们的复位（初始化）将由软件完成。

本书中的设计建议只考虑复位信号是同步复位的情况。对于 CPU 这样的数字逻辑电路来说，复位信号是采用同步复位还是异步复位，就如同考虑大尾端好还是小尾端好一样，是一个没有必要争论的问题，每个用户按照自己的喜好选择就可以。但是切记，一旦决定了，那么在一个设计中就只能^①使用所选择的那种方式。

最后，我们要讨论一个初学者最容易犯的错误——在复位信号有效期间就对外发起

^① 事实上，更严谨的措辞应该是“尽可能”，但是这就会导致一个度的把握问题，对于初学者来说，或者在本书所涉及的实践范围内，我们给出更严格的限制以降低难度。

第一条指令的取指请求。就目前我们所接触的设计场景来说，这种设计是可行的，因为指令 RAM 多读一次或少读一次都不会产生副作用。但是，在绝大多数真实系统中，处理器核复位后的第一条指令不是从内部的指令 RAM 取回来的，而是从处理器核外部的片上 ROM 甚至是 CPU 芯片外部的 Flash 芯片中取回来的。处理器核访问这些 ROM 或 Flash 通常是通过发起（片上）总线访问请求来实现的。处理器核通常是整个 CPU 芯片上较晚结束复位的部件，所以在处理器核硬件复位的后期，外部总线和设备模块大多已经复位结束，并进入可以工作的状态，如果这个时候处理器核已经开始持续向外部总线发起访问请求，总线和外设就会响应这些访问请求，但是此时处理器核却还在复位过程中。等到复位撤销后，处理器核所认为的外部总线的工作状态和总线实际的工作状态会发生不一致，进而导致出错。

5.2 指令相关与流水线冲突

现在我们进入简单流水线 CPU 设计的第二阶段。我们将在第一阶段设计的基础上，分析指令相关对流水线 CPU 设计带来的影响，进而对我们的 CPU 设计进行调整。有关指令相关和流水线冲突的原理性阐述，请参考《计算机体系结构基础》（第3版）的9.3节或其他相关文献。这里略去分析过程，总结几个要点如下：

- 1) 单发射静态流水线上只需要解决围绕寄存器的写后读数据相关所引发的流水线冲突，围绕寄存器的写后写和读后写数据相关不会引发流水线冲突，围绕内存的所有类型的数据相关也不会引发流水线冲突。
- 2) 对于单发射静态流水线 CPU，如果所实现的指令集没有转移延迟槽，那么由于转移指令最早也只能在译码阶段进行处理（执行），在没有实现诸如分支目标缓存（Branch Target Buffer, BTB）的情况下，必须考虑控制相关所引发的流水线冲突。
- 3) 单发射静态流水线上的结构相关所引发的流水线冲突的处理可以简化为：后一级流水线的指令无法运行，则前一级流水线的指令也无法运行。

针对第3点，前面给出的流水线 CPU 设计中所采用的逐级互锁控制机制能够确保任何一级流水线指令无法运行时，它前面的各级流水线的指令都可以及时停止，同时不丢失数据，所以结构相关引发的流水线冲突也已经得到解决。

我们接下来主要解决寄存器写后读数据相关所引发的流水线冲突以及控制相关所引发的流水线冲突。

5.2.1 处理寄存器写后读数据相关引发的流水线冲突

产生由寄存器写后读数据相关所引发的流水线冲突的场景是：产生结果的指令（“写者”）尚未将结果写回到通用寄存器堆中，而需要这个结果的指令（“读者”）已经在译码阶段了，此刻它从通用寄存器堆中读出的数值是旧值而非新值。

如何避免产生这种错误呢？一种直观的解决思路是：让需要结果的指令在译码阶段一直等待，直到产生结果的指令将结果写入到通用寄存器堆，才可以进入到下一级的执行阶段中。这一节我们就采用这种思路来调整设计。

这套设计思路中的关键点在于控制译码阶段指令前进还是阻塞的条件如何生成，其核心是判断处于流水线不同阶段的指令是否存在会引发冲突的“写后读”相关关系。结合我们设计的五级流水线 CPU 的结构，这个判断可以具体描述为：处于译码阶段的指令具有来自非 0 号寄存器的源操作数，那么如果这些源操作数中任何一个的寄存器号与当前时刻处于执行阶段、访存阶段或写回阶段的指令的目的操作数的寄存器号（非 0 号）相同，则表明处于译码阶段的指令与执行阶段、访存阶段或写回阶段的指令存在会引发冲突的“写后读”相关关系。

在上面给出的具体描述中，有些读者可能不理解为什么译码阶段指令的源寄存器号也要和写回阶段指令的目的寄存器号进行比较。出现这个问题的原因是，没有正确理解寄存器堆同时读写同一项时读出数据的行为。请回想一下第 3 章实践任务 2 中所分析的寄存器堆的行为，在写使能有效的这个时钟周期（写使能将被这个时钟周期与下一个时钟周期之间的上升沿采样），读端口的地址若和写地址一样，那么读出数据的端口上只能出现这一项的旧值而非此时写端口的写入数据。只要理解这个时序特性，自然就会知道译码阶段的指令与写回阶段的指令是要比较寄存器号的。

在把前面给出的描述落实到具体代码实现的过程中，还有一个隐含的细节容易被初学者忽视，那就是一定要保证被比较的两个寄存器号都是有效的。这体现为三个方面。第一，参与比较的指令到底有没有寄存器的源操作数或是寄存器的目的操作数。比如，`addi.w` 指令只有 `rj` 一个寄存器的源操作数，没有 `rk` 寄存器的源操作数。再比如，`bl` 指令压根就没有寄存器的源操作数，`beq` 和 `bne` 这两条转移指令则没有寄存器的目的操作数。第二，如果指令的定义确实有寄存器的源操作数或目的操作数，但是寄存器号为 0，那么也不用进行比较。因为在 LoongArch 架构下，0 号寄存器的值恒为 0，所以对这个寄存器存在的相关关系不需要做特殊处理。第三，用来进行比较的阶段上到底有没有指令，如果没有指令，那么这一阶段流水线缓存中的寄存器号、指令类型等信息都是

无效的。如果在写比较逻辑的时候没有考虑到这些特例，那么写出来的处理器虽然不会出错，但是会在某些情况下出现不必要的阻塞，导致性能损失。

完成了会引发冲突的“写后读”相关关系的判断后，最后一个步骤就是在判断条件成立的时候，把这条指令阻塞在译码阶段。还记得我们每一阶段流水线的 `ready_go` 信号吗？在这里，只需要对译码阶段的 `ready_go` 信号进行调整就可以了。显然，它不是恒为 1，如果发现译码阶段的指令与后面执行、访存、写回三个阶段的指令间存在会引发冲突的“写后读”相关关系，那么就要把 `ready_go` 信号置为 0。

5.2.2 处理控制相关

我们在前面进行 CPU 的流水线切分的时候，已经将转移指令操作中与 `nextPC` 相关的处理逻辑（判断是否跳转、跳转目标）放在译码阶段，因此处理控制相关的关键设计点就是：取消可能处在取指阶段的、错误执行路径上的指令，不能让这条错误取回的指令进入到 CPU 流水线中执行。

为了理解为何进行上述处理就能解决控制相关问题，我们建议读者：

- 1) 回顾一下同步读 RAM 的时序特性，进而明确指令 RAM 的读请求和读地址是在 `pre-IF` 阶段发出的，指令处于取指阶段时已经是在等待指令 RAM 返回读数了；
- 2) 画一个流水线时空图来帮助理解。

有了这些准备，我们接下来将“推演”一下转移指令在已有的五级流水线 CPU 上的执行情况。假设有一条转移指令，其 PC 是 `0x1000`。在第 1 个时钟周期，它位于取指阶段，此时 `pre-IF` 阶段同时在计算 `nextPC` 了。此时 `nextPC` 是不可能根据处在取指阶段的转移指令的情况来设置的，因为此时转移指令的指令码刚刚从指令 RAM 中读出，CPU 还不知道这条指令是不是转移指令，更不知道它是否跳转以及跳转到哪里，这两方面信息都是在译码阶段处理后才能获得的。所以，在第 1 个时钟周期，`nextPC` 只能是按照顺序取指这一默认情况置为 `0x1004`。在第 2 个时钟周期，PC 为 `0x1000` 的转移指令来到译码阶段，**PC 为 `0x1004` 的指令来到取指阶段**，如果转移指令在译码阶段处理后发现自己并不跳转，那么 `pre-IF` 阶段继续顺序发出 `PC=0x1008` 的取指请求，并且取指阶段的 `PC=0x1004` 的指令也是程序期望执行路径上的指令，它将留在流水线继续执行；如果转移指令在译码阶段处理后发现自己需要跳转（假设跳转目标地址为 `0x2000`），那么即使立刻将 `pre-IF` 阶段生成的 `nextPC` 置为 `0x2000`，也无法改变 `PC=0x1004` 的指令处在取指阶段的现状了，很显然 `PC=0x1004` 的指令不是程序期望执

行路径上的指令，它是不能被执行的（否则会错误地修改处理器的状态）。怎么对待这个溜进来的“不速之客”呢？一种方式是让它不要惹麻烦（即一些教科书中给出的将该指令处理成 nop 指令的解决方案），另一种方式是把它请出去（即我们这里建议的将该指令取消掉的方案）。

我们不推荐将误取指令处理成 nop 指令的方案。缺点有二：其一，这条 nop 指令不能被标记上任何异常，否则将违反精确异常的要求^①，要处理好这一点，就要把这种硬件自动转换的 nop 指令和程序中正常的 nop 区分开，徒增负担；其二，将指令转换成 nop 指令需要修改 32 根线，与我们推荐的方案相比，控制信号的扇出变大了，延迟会变差。此外，还要提醒一下，LoongArch 指令集中全 0 编码不是 nop 指令，不要采用把指令码全抹 0 的方式来置 nop 指令。

我们为什么推荐将指令取消掉的方案呢？请读者回忆一下我们给出的流水线设计参考中，是如何表示流水线上有没有指令的。我们是通过各级流水线的那个 valid 位来实现的。所以，取消一条指令，就是把**伴随着这条指令的** valid 抹成 0 就可以了。请注意，这里我们强调的是“伴随着这条指令的 valid”，而没有说是取指阶段的 valid。我们来看下面初学者常犯的一种错误。假设在没有考虑跳转转移指令取消的情况下取指阶段的 valid 的代码如下：

```
always @(posedge clock) begin
    if (reset)
        fs_valid <= 1'b0;
    else if (fs_allowin)
        fs_valid <= to_fs_valid;
end
```

假设输入到取指阶段的转移指令跳转取消信号是 br_taken_cancel，于是有的初学者是这样实现取消误取入指令功能的：

```
always @(posedge clock) begin
    if (reset)
        fs_valid <= 1'b0;
    else if (br_taken_cancel)
        fs_valid <= 1'b0;
    else if (fs_allowin)
        fs_valid <= to_fs_valid;
end
```

我们猜测写出上面的代码的思路应该是，需要被取消的指令在取指阶段，所以当

① 有些读者可能对异常相关的内容还不理解，可以在后面增加异常支持的实验中回过头来思考一下。

br_taken 有效时需要将其清 0。上面的代码是错误的。读者可以试着推演一下，这段代码对应的电路逻辑在位于取指阶段的指令没有被阻塞的情况下，非但不能将误取入的指令取消掉，而且还可能导致转移指令跳转目标处的指令没有取入到流水线中。如果对照比较我们上面强调的“将伴随着误取入指令的 valid 置 0”的设计方案，应该就能看出问题出在哪里了。在误取入指令没有被阻塞在取指阶段时，下一个时钟周期，它将来到达译码阶段，所以我们需要把从取指阶段到译码阶段的 valid 置为 0 而不是将进入取指阶段的 valid 置为 0。所以，应该对译码阶段的 valid 写入逻辑进行调整：

```
always @(posedge clock) begin
    if (reset)
        ds_valid <= 1'b0;
    else if (br_taken_cancel)
        ds_valid <= 1'b0;
    else if (ds_allowin)
        ds_valid <= fs_to_ds_valid;
end
```

不过，仅是上面这样的代码调整就完备了吗？在目前的场景中，仅这样改是对的。因为指令 RAM 总是能在读请求发出的下一个时钟周期返回读数据，所以位于取指阶段的指令肯定不会因为等待指令取回而停留，那么当位于取指阶段的误取入指令要进入译码阶段时，br_taken_cancel 一定还是有效的。（这个推演请读者自行完成。）但是，一旦取指不会立即返回，即位于取指阶段的指令可能因为等待指令取回而停留时，仅调整 ds_valid 的写入逻辑就不够了，还需要同时调整 fs_valid 的写入逻辑，具体修改如下：

```
always @(posedge clock) begin
    if (reset)
        fs_valid <= 1'b0;
    else if (fs_allowin)
        fs_valid <= to_fs_valid;
    else if (br_taken_cancel)
        fs_valid <= 1'b0;
end
```

看上去这个改动和之前那个有错的改动很像，只不过把 br_taken_cancel 清 0 这个 if 分支挪到下面去了。这看似简单的位置调整后含义却发生了很大的变化。我们将最后一行 fs_valid 清 0 的条件展开来写，就是 !reset && !fs_allowin && br_taken_cancel，而 !reset && !fs_allowin 蕴含着 fs_valid && (!fs_ready_go || !ds_allowin)，这句代码用自然语言解读一下就是“当取指阶段有一条指令且无法在下一个时钟周期进入下一流

水线阶段”。在这种情况下，如果有转移指令发生跳转，那么需要将取指阶段的指令取消。

最后再补充一个关于 `br_taken_cancel` 信号有效时的注意事项。由于转移指令可能需要对源操作数进行运算处理后才能得到是否跳转以及跳转目标的提示，所以这类指令即使到达了译码阶段，也可能因为存在寄存器的写后读相关而被阻塞住，那么在其获得正确的源操作数之前（也即写后读相关阻塞解除之前），**一定不能**将 `br_taken_cancel` 信号置为有效。

至此，我们就完成了一个能够正确处理各类相关所引发流水线冲突的 CPU。尽管它在某些时候性能还有些偏低，但是至少能够正确地执行一个正常的 LoongArch 可执行程序了。

5.3 流水线数据前递设计

现在我们进入简单流水线 CPU 设计的最后一个阶段。我们将继续针对流水线 CPU 中“写后读”相关所引起的冲突的处理，探讨一种更高性能的解决方案——**前递**（forward）技术。

我们介绍了用阻塞译码级指令的方式来解决寄存器“写后读”相关所引发的流水线冲突。读者在实践任务 8 中应该会观察到一个现象，那就是需要结果的指令在译码级等待的过程中，前面产生结果的指令其实已经生成出结果了，只不过是还没有写入到寄存器堆中。有没有可能让前面的指令直接把已经生成出来的结果直接转给后面的指令，这样后面的指令就不再需要等待了吗？答案是可以的。基于这个思路设计的就是流水线 CPU 的前递技术，也叫**旁路**（bypass）技术。基本原理不难理解，可参看《计算机体系结构基础》（第 3 版）的 9.3.1 节或其他相关文献。接下来对具体如何设计展开分析。我们仍采用先设计数据通路再处理控制信号的设计流程。

5.3.1 前递的数据通路设计

5.3.1.1 运算结果的前递路径

我们先来看产生结果的指令如果是 `add.w` 的话，该如何处理。`add.w` 指令执行流水线产生的结果，原先只能进入到访存级流水线缓存中，因此需要增加一条专门的路径，将结果直接传递给后面的指令。那么是不是只增加这一条前递路径就够了呢？显然不

够。当 add.w 指令位于访存级流水线时，它的结果目前只能进入写回级流水线缓存中，所以还要增加另一条专门的路径用于结果的传递，否则后面的指令还将因无法及时获得结果而陷入等待状态。继续分析下去，我们还会发现 add.w 指令位于写回级流水线的时候，也需要添加另一条独立的前递路径。

在考虑前递路径设计时，add.w 指令代表那些在执行级就能产生结果的指令。但是 ld.w 指令不属于这一类指令，它直到访存级才能生成结果。不过分析之后我们发现 ld.w 指令完全可以复用那些为了前递 add.w 类指令结果而在访存级和写回级增加的前递路径。因为前递路径只是把结果送出去，有寄存器号和数值这两个信息就够了，与产生结果的指令具体是做什么的没有关系。

上述前递路径落实到 CPU 的结构设计框图上，体现为 3 条带箭头的连线。那么各条连线的起点和终点在哪里？我们先来看执行流水级所产生的结果如何前递。先说两种初学者中常见的错误设计方案。

第一种错误方案：起点位于执行级 ALU 的结果输出处，终点位于执行级 ALU 输入数据生成逻辑处。

第二种错误方案：起点位于访存级流水线缓存中存放执行级 ALU 结果的触发器的 Q 端口输出处，终点位于译码级寄存器堆读出结果生成逻辑处。

如果在结构图上画出这两种设计方案的前递路径，你会立刻发现它不正确。对于错误方案一，电路中居然出现了一条从 ALU 的输出连接到 ALU 输入的组合环路。对于错误方案二，位于执行级的指令根本拿不到它前一条指令的结果。为什么有些人会犯这么直观的错误呢？往往是因为他们没有遵循“先设计再编码”的步骤，还没有想清楚设计就急急忙忙地去改代码，导致出现了一种逻辑混乱的设计。所以这里再强调一遍，不要轻视设计环节，设计时不要图省事不画结构设计图。

这里给出两种可行的设计方案。

方案一：起点位于执行级 ALU 的结果输出处，终点位于译码级寄存器堆读出结果生成逻辑处。

方案二：起点位于访存级流水线缓存中存放 ALU 结果的触发器的 Q 端口输出处，终点位于执行级 ALU 输入数据生成逻辑处。

两种方案在功能上都是对的。读者可以用“add.w r2, r1, r1 ; sub.w r3, r2, r2”这两条指令的序列在流水线上推演一下，将会发现当 sub.w 指令紧跟着 add.w 指令在流水线中前进时，它在 ALU 中进行计算时源操作数 r2 的值一定是 add.w 指令算出的结果值。我们把方案一的特点总结为“流水级组合逻辑结果前递到译码级寄存器读出”，

把方案二的特点总结为“流水线缓存保存的前一级结果前递到执行级 ALU 输入”，两种方案的一个重要区别在于终点是在译码级的组合逻辑末端还是在执行级的组合逻辑始端。

哪种方案好呢？现在还看不出来，我们需要沿着两套方案的思路继续考虑余下的前递路径的起点和终点位置。对于方案一，访存级结果的前递路径起点就是数据 RAM 返回结果和访存级缓存所存的 ALU 结果经过二选一之后的结果输出处，写回级结果的前递路径起点就是写回级将要写入到寄存器堆中的结果处。一切都很和谐。对于方案二，访存级结果的前递路径起点是写回级流水线缓存中存放访存级流水线执行结果的触发器的 Q 端输出处，写回级结果的前递路径……是不是突然发现写回级的结果没有路径可以前递了？如果这一拍写回级是“`add.w r2, r1, r1`”指令，译码级是“`sub.w r3, r2, r2`”指令，执行和访存级上没有写 r2 寄存器的指令。那么除非你将前递路径的起点放在写回级将要写入到寄存器堆中的结果处，终点放到译码级寄存器堆读出结果生成逻辑处，否则下一拍“`sub.w r3, r2, r2`”指令是不能进入到执行级的。为了尽可能多地通过前递来消除指令阻塞带来的性能损失，我们会发现无论哪套方案，都需要有前递路径的终点在译码级寄存器堆读出结果生成逻辑处。那么以个人的喜好而言，我们倾向于方案一，这样所有的前递路径终点一致，使得由前递结果生成正确的寄存器源操作数的逻辑可以在一个地方集中处理。

如果说上面选择方案一多少还带有一些主观色彩，那么我们再来看另一个情况。还记得为了解决控制相关引发的流水线冲突，我们将所有转移指令的处理都放到译码级了吗？目前实现的转移指令中，`beq`、`bne`、`jirl` 三条都有寄存器源操作数。如果它们和它们前面的指令存在“写后读”的相关关系，那么除非前递路径的终点是在译码级，否则它们就必须阻塞自己直到结果写到寄存器堆中。分析到这儿，是不是觉得选择方案一更合理一些了。以我们的经验来看，对于初学者，方案一实现起来遇到的“陷阱”要少一些，所以接下来的介绍将只基于方案一展开。

5.3.1.2 前递结果的多路选择

前递路径构建好了，整个流水线 CPU 前递的数据通路设计还有一个尾巴，那就是译码级产生寄存器读结果的逻辑该怎么调整。从空间上看，译码级指令的寄存器源操作数 1 的数值，可以来自通用寄存器堆读端口 1 的输出，也可以来自执行级、访存级、写回级三处前递来的结果；同样，译码级指令的寄存器源操作数 2 的数值，可以来自通用寄存器堆读端口 2 的输出，也可以来自执行级、访存级、写回级三处前递来的结果。所

以我们需要添加两个“四选一”，而且是两个具有选择优先级的“四选一”。为什么这里的“四选一”需要有优先级？我们来看一个极端的例子，对于指令序列“add.w r4, r1, r1; add.w r4, r2, r2; add.w r4, r3, r3; sub.w r6, r5, r4”，请问当“add.w r4, r1, r1”位于写回级、“add.w r4, r2, r2”位于访存级、“add.w r4, r3, r3”位于执行级时，此时位于译码级的“sub.w r6, r5, r4”的源操作数 2 应该选择哪个数值？很显然应该选择执行级前递过来的结果。所以你会看到选择哪个前递结果，不仅要看源操作数的寄存器号与前递来的结果的寄存器号是否一致，还要考虑不同级流水线之间的优先级关系。

这里插段题外话。在前面的分析过程中，读者已经看到我们会假设出一段指令序列，然后推演它们在所设计的流水线 CPU 中的执行情况，以此来对不同的设计预案进行对比评估，进而得到一个更简洁、更合理的设计方案。读者也要逐步掌握这种方法。还记得前面我们反复强调的电路设计要做到“谋定而后动”吗？那么设计 CPU 的时候具体怎么“谋”呢？就是用这里介绍的假想指令序列进行执行过程推演的方式完成的。为什么要用“思维推演”的场景而不是采用验证用例体现的场景呢？因为任何基于测试用例的验证方式，最大的风险都在于验证激励覆盖的情形是否完备。尽管验证人员会有一系列技术和工程手段来降低这个风险，但是作为设计者，尤其是结构设计者，我们不能把设计的正确性完全寄托在验证用例是否完备上。结构设计人员在设计时对各种可能情况进行严谨推演，是保证最终设计正确可靠的“第一把锁”，而验证人员的验证工作则是保证设计正确可靠的“第二把锁”。我们要上“双保险”！

5.3.2 前递的流水线控制信号调整

到目前为止，我们已经把前递相关所新增的数据通路以及通路中“四选一”部件的控制信号都设计完毕了。如果按照这个设计对具有阻塞功能的 CPU 进行改造，然后运行所有测试用例，你会高兴地看到测试都通过了。但是，很不幸地告诉你，此时你设计的这个带前递功能的流水线 CPU 并不正确。因为你会发现同样的程序，在这个带前递功能的流水线 CPU 上运行，时间和原先只具有阻塞功能的 CPU 完全一样。我们引入前递技术不就是为了提升性能吗？费了半天劲，性能一点没改善，岂不是白忙活了？问题出在哪里？是译码级的阻塞控制信号还没有修改。它还是以最严格的方式在阻塞。那么是不是大笔一挥，直接将译码级的 ready_go 改成恒为 1 呢？请冷静。看这样一个指令序列：“ld.w r2, r1, 0x0; add.w r4, r3, r2”。如果在这一拍，ld.w 指令位于执行级，add.w 指令位于译码级，请问 add.w 指令下一拍能进入执行级吗？显然不能，因为此刻位于执行级的 ld.w 指令没有生成最终的结果，所以纵使你有前递的通路，但是没有正确的

数据，`add.w` 指令就必须在译码级停着，等到前面的 `ld.w` 指令到达访存级时，它才能通过访存级至译码级的前递通路拿到正确的数值，从而在下一拍进入到执行级。所以碰到这种情况，即译码级的指令与当前位于执行级的 `ld.w` 指令在写后读相关时，译码级的 `ready_go` 信号还是要置为 0。在目前 CPU 只实现了 20 条指令的情况下，引入前递技术之后，译码级的阻塞条件仅有这种情况。随着后面实现指令的不断增多，读者一定要及时关注 `ready_go` 置为 0 的条件是否需要调整。

5.3.3 前递引发的主频下降

这一节给大家介绍的流水线 CPU 的前递设计方案比较容易正确实现而且能够最大限度地减少因为写后读相关而引起的流水线阻塞。但是天下没有免费的午餐，这套设计方案的一个不足之处在于它会增加 CPU 关键路径的延迟，导致 CPU 主频下降。我们接下来给大家分析一下，目的是向各位展示如何分析设计中的关键路径延迟。

CPU 前递的设计改动就是译码级修改了 `ready_go` 信号的生成逻辑、增加了几组连线、增加了两个“四选一”。对于第一处改动，`ready_go` 信号的生成逻辑是改简单而非改复杂了，所以这里的修改不应该造成主频下降。对于第二处改动，执行、访存、写回三级流水线的组成逻辑结果增加了旁路确实会增加这些逻辑最后一两级逻辑器件的输出负载，由此导致这些逻辑器件的延迟增加，但是目前的设计规模很小，新增走线的距离也没那么远，所以这里的延迟增加应该不是主频下降的主要原因。所以译码级增加的那两个“四选一”部件就成为我们的重点怀疑对象了，但是当阅读最终的时序报告时，我们发现关键路径延迟的增加量可不止一个“四选一”部件的延迟。因此这个时候我们基本上能判定，一定是这两个“四选一”部件参与并引入了一条新的延迟特别长的组合逻辑路径。

于是我们回到设计，梳理途经这两个“四选一”部件的所有组合逻辑路径，答案就逐渐浮出水面了。这条新出现的路径是：执行级流水线缓存的触发器 Q 端 → ALU → 前递路径 → 译码级四选一 → 转移指令跳转方向的判定逻辑 → `GenNextPC` 的四选一 → 取指请求的虚实地址转换 → 指令 RAM 的地址输入端口。

我们希望读者掌握上面这种基于梳理设计而不仅仅是看 EDA 工具提供的时序报告来确定设计中关键路径的方法。时序报告要看，但那只是给你一个提示和反馈，从设计出发进行分析才是正道。我们并不将上述关键路径的时序优化作为本书的指导目标，因此接下来也只是简单地提供一些解决思路供学有余力的读者参考。

完全舍弃前递，主频最高，流水线效率低；最大限度地进行前递，流水线效率最

高，但主频低。这里进行性能优化要在主频和流水线效率之间寻求一个平衡点。通过分析，我们发现矛盾的焦点在于对转移指令进行前递了。再进一步分析，我们又发现如果不是直接把 ALU 的组合逻辑输出结果前递给译码级的转移指令，而是将存下来的 ALU 结果（在访存级流水线缓存中）传递过去，那么也能够消除关键路径。这样我们只是在有些情况下将转移指令阻塞一拍，如果所运行的程序中这种情况出现并不频繁，那么程序在流水线上的整体执行效率并不会下降多少，同时又由于 CPU 的主频得到提升，所以整体性能就得到了提升。

5.4 CPU 设计实验功能仿真调试技术进阶

本节介绍流水线 CPU 调试该抓哪些信号以及如何看这些信号。

5.4.1 valid 和 PC 信号不能少

从前面的内容可知，调试 CPU 的时候必须要结合测试程序的指令序列进行。我们通过阅读测试程序及其反汇编代码来了解 CPU 所执行的指令序列。如何快速地将它们与抓取的信号联系起来呢？核心纽带就是指令的 PC。这也是为什么现阶段设计的 CPU 还没有支持异常功能，而我们仍然建议将 PC 从取指阶段沿着各级流水线一路传递下去。因为这样每一级流水线的缓存中都会保存有这一级流水线当前正在处理的指令的 PC，将各级的 PC 信号抓到波形窗口中，就可以一目了然地看出一条指令是如何在流水线中前进的。

除了各级的 PC 信号外，在采用本书推荐风格的代码中，每一级流水线的 valid 位也是要抓出来的。对于任意一级流水线，不要在其 valid 为 0 的周期内分析该级流水线的其他信号，除非你的设计意图是希望这个周期 valid 为 1 而你正在分析为什么它没有置为 1。

其余较为常抓的信号有：指令在写回级写通用寄存器堆的使能、地址、数据，指令 RAM 和数据 RAM 访问接口上的信号，以及取指或译码级上的指令码。不过这些信号并不一定每次都要抓出来，要视你调试的需求而定。

5.4.2 各流水线信号分组有序摆放

对于流水线 CPU，强烈建议大家将同一级流水线的信号放在同一组中，然后按照

各级流水线的顺序将各组自上而下或自下而上依次摆放。

随着设计逐步成熟固化以及大家调试经验的积累，建议将各级流水线基础的、常用的信号抓出来，按照上面的建议分好组并摆放好，然后将这些分组摆好的基本信号存成信号文件。这样再重新启动仿真时，仿真一结束就可以把这些基本信号显示出来，节省了再添加一遍的时间。

5.4.3 先遍历指令再遍历流水线

假定信号波形都按照上面建议的那样摆放好了，那么这样花花绿绿的一片又该怎么看呢？前面的调试方法介绍中我们说过，得先找到源头错误对应的那条指令，再分析这条指令具体错在电路的哪个部分。

第一步定位指令的过程，建议你只盯着流水线的一级来看，通常我们选择写回级。根据写回级的 valid 和 PC 信号，结合反汇编中查明的指令序列，在波形中前后移动以确定出错指令。再提醒一下，如果移动跨度超过视界，可以考虑用查找信号值来加快移动速度和精确度。

找到可疑指令之后，根据它的 PC 看反汇编代码，明确它是写通用寄存器堆的、写内存的，还是分支跳转类的。如果是写通用寄存器堆的，那就从这条指令在写回级时访问寄存器堆的信号开始查看；如果是写内存的，那就从这条指令在执行级时访问数据 RAM 的信号开始查看；如果是分支跳转类的，那就从这条指令在译码级时生成出的影响 nextPC 的 taken 和 target 信号开始查看。之后再查看什么信号就按照 CPU 设计中的数据通路逆向一路追查下去。

当一条可疑指令在 CPU 的执行过程中被逆向梳理一遍以后，要么已经能够明确 CPU 设计中存在的错误，要么会发现这条指令还不是错误的源头，那就回到定位指令的过程继续沿时间轴向前或向后定位指令。

5.5 任务与实践

完成本章的学习后，希望读者能够完成以下 3 个实践任务：

- 1) 不考虑相关引发的冲突的简单流水线 CPU，参见 5.5.1 节。
- 2) 阻塞技术解决相关引发的冲突，参见 5.5.2 节。
- 3) 前递技术解决相关引发的冲突，参见 5.5.3 节。

5.5.1 实践任务 7：不考虑相关引发的冲突的简单流水线 CPU

本实践任务要求在实践任务 6 实现的单周期 CPU 基础上完成以下工作：

- 1) 调整 CPU 顶层接口，增加指令 RAM 的片选信号 `inst_sram_en` 和数据 RAM 的片选信号 `data_sram_en`。
- 2) 调整 CPU 顶层接口，将 `inst_sram_we` 和 `data_sram_we` 都从 1 位的写使能调整为 4 位的字节写使能。
- 3) 设计一个不考虑相关引发的冲突的单发射五级流水线 CPU。
- 4) 运行 `exp7` 对应的 `func`，要求成功通过仿真和上板验证。

请参照 2.3.1 节介绍的方式获取本次实践任务所需的实验开发环境。具体的实验环境仍位于 `mycpu_env/` 目录下，不过验证时不再使用 `soc_dram/` 子目录，而应使用 `soc_bram/` 子目录。

伴随着 CPU 访问的指令 RAM 和数据 RAM 的实现形式从 distributed RAM 更换为 block RAM，实验开发环境也需要有所调整：仍然是 `mycpu_env` 实验环境，`gettrace/`、`func/` 和 `myCPU/` 子目录的位置和用途依然维持不变，只是 `soc_verify/` 子目录下不再使用 `soc_dram/` 子目录而是改为使用 `soc_bram/` 子目录，`soc_bram/` 子目录中的文件组织结构和用途与 `soc_dram/` 子目录中的相似。调整后的实验开发环境的目录结构及各部分功能简介如下：

<code>--gettrace/</code>	生成参考 <code>trace</code> 的部分。
<code>--func/</code>	实验任务所用的功能验证测试程序。
<code>--myCPU/</code>	自己实现的 CPU 的 RTL 代码。
<code>--soc_verify/</code>	自己实现的 CPU 的 SoC 系统验证环境。
<code>--soc_bram/</code>	CPU 对外连接 block RAM 接口时对应的验证环境。
<code>--rtl/</code>	SoC_Lite 设计代码目录。
<code>--soc_lite_top.v</code>	SoC_Lite 的顶层文件。
<code>--CONFREG/</code>	<code>confreg</code> 模块，用于访问 CPU 与开发板上的数码管、拨码开关等外设。
<code>--BRIDGE/</code>	1×2 的桥接模块，CPU 的 <code>data_sram</code> 接口同时访问 <code>confreg</code> 和 <code>data_ram</code> 。
<code>--xilinx_ip/</code>	定制的 Xilinx IP，包含 <code>clk_pll</code> 、 <code>inst_ram</code> 、 <code>data_ram</code> 。
<code>--testbench/</code>	功能仿真验证平台。
<code>--mycpu_tb.v</code>	功能仿真顶层，该模块会抓取 <code>debug</code> 信息与 <code>golden_trace.txt</code> 进行比对。
<code>--run_vivado/</code>	Vivado 工程的运行目录。
<code>--constraints/</code>	Vivado 工程的设计约束。
<code>--mycpu_bram_prj/</code>	Vivado 工程文件所在目录。

从实践任务 7 开始的指令 RAM 和数据 RAM 均采用 block RAM 实现，访问时需要给出片选信号。为此 `myCPU` 顶层接口中增加指令 RAM 的片选信号 `inst_sram_en` 和数据 RAM 的片选信号 `data_sram_en`。两个信号均为 1 位，均为高电平有效。

尽管实践任务 7 中存数指令仅实现了 `st.w`, 但考虑后续实践任务的需求, `myCPU` 顶层接口中的 `inst_sram_we` 和 `data_sram_we` 都从 1 位改为 4 位, 其含义也从 RAM 的写使能调整为 RAM 的字节写使能。

实验环境准备就绪后, 请参考下列步骤完成本实践任务:

- 1) 将所实现 CPU 的代码更新至 `mycpu_env/myCPU/` 目录中。
- 2) 修改 `func` 配置文件——`mycpu_env/func/include/test_config.h`, 选择 `exp7` 的配置, 编译。(如果是通过压缩包 `exp7.zip` 获取实验开发环境的, 请跳过该步骤。)
- 3) 打开 `gettrace` 工程——`mycpu_env/gettrace/gettrace.xpr`。(该 Vivado 工程中的 IP 核是使用 Vivado 2019.2 创建的, 如果使用更高版本的 Vivado 打开, 请参考附录 D.4 节进行 IP 核升级。)运行 `gettrace` 工程的仿真(进入仿真界面后, 直接点击 `run all` 等待仿真运行完成), 生成新的参考 `trace` 文件 `golden_trace.txt` (`mycpu_env/gettrace/golden_trace.txt`)。要等仿真运行完成, `golden_trace.txt` 才有完整的内容。(如果是通过压缩包 `exp7.zip` 获取实验开发环境的, 请跳过该步骤。)
- 4) 进入 `mycpu_env/soc_verify/soc_bram/run_vivado/` 目录下启动验证 `myCPU` 的工程。如果该目录下尚未创建工程, 请参照附录 D.2 节介绍的步骤, 利用该目录下的 `create_project.tcl` 文件创建工程。如需要, 请参考附录 D.4 节进行 IP 核升级。
- 5) 参考 4.4.5.2 节, 对工程中的 `inst_ram` 重新定制。(如果是通过压缩包 `exp7.zip` 获取实验开发环境的, 请跳过该步骤。)
- 6) 在验证 `myCPU` 的工程中运行仿真(进入仿真界面后, 直接点击 `run all`), 进行功能验证与调试, 直至仿真测试通过。
- 7) 在验证 `myCPU` 的工程中综合实现后生成二进制码流文件, 进行上板验证。(如果无硬件实验平台, 请跳过该步骤。)

5.5.2 实践任务 8: 阻塞技术解决相关引发的冲突

本实践任务要求在实践任务 7 实现的 CPU 基础上完成以下工作:

- 1) 使用阻塞和取消技术处理寄存器写后读数据相关和控制相关所引发的流水线冲突。
- 2) 运行 `exp8` 对应的 `func`, 要求成功通过仿真和上板验证。

请参照 2.3.1 节介绍的方式获取本次实践任务所需的实验开发环境。具体的实验环

境仍位于 mycpu_env/ 目录下，且仍使用 soc_bram/ 子目录。

实验环境准备就绪后，请参考下列步骤完成本实践任务：

- 1) 将所实现 CPU 的代码更新至 mycpu_env/myCPU/ 目录中。
- 2) 修改 func 配置文件——mycpu_env/func/include/test_config.h，选择 exp8 的配置，编译。（如果是通过压缩包 exp8.zip 获取实验开发环境的，请跳过该步骤。）
- 3) 打开 gettrace 工程——mycpu_env/gettrace/gettrace.xpr。（该 Vivado 工程中的 IP 核是使用 Vivado 2019.2 创建的，如果使用更高版本的 Vivado 打开，请参考附录 D.4 节进行 IP 核升级。）运行 gettrace 工程的仿真（进入仿真界面后，直接点击 run all 等待仿真运行完成），生成新的参考 trace 文件 golden_trace.txt（mycpu_env/gettrace/golden_trace.txt）。要等仿真运行完成，golden_trace.txt 才有完整的内容。（如果是通过压缩包 exp8.zip 获取实验开发环境的，请跳过该步骤。）
- 4) 进入 mycpu_env/soc_verify/soc_bram/run_vivado/ 目录下启动验证 myCPU 的工程。如果该目录下尚未创建工程，请参考附录 D.2 节介绍的步骤，利用该目录下的 create_project.tcl 文件创建工程。如需要，请参考附录 D.4 节进行 IP 核升级。如果该目录下已有前一实践任务创建过的工程，可以在打开工程后，参照附录 D.3 节介绍的步骤，更新项目中 CPU 实现文件的列表。
- 5) 参考 4.4.5.2 节，对工程中的 inst_ram 重新定制。（如果是通过压缩包 exp8.zip 获取实验开发环境的，请跳过该步骤。）
- 6) 在验证 myCPU 的工程中运行仿真（进入仿真界面后，直接点击 run all），进行功能验证与调试，直至仿真测试通过。
- 7) 在验证 myCPU 的工程中综合实现后生成二进制码流文件，进行上板验证。（如无硬件实验平台，请跳过该步骤。）

5.5.3 实践任务 9：前递技术解决相关引发的冲突

本实践任务要求在实践任务 8 实现的 CPU 基础上完成以下工作：

- 1) 加入适当的数据前递通路来减少阻塞。
- 2) 运行 exp9 对应的 func，要求成功通过仿真和上板验证，并且仿真运行时间较 exp8 的结果有下降。

请参照 2.3.1 节介绍的方式获取本次实践任务所需的实验开发环境。具体的实验环境仍位于 mycpu_env/ 目录下，且仍使用 soc_bram/ 子目录。

实验环境准备就绪后, 请参考下列步骤完成本实践任务:

- 1) 将所实现 CPU 的代码更新至 mycpu_env/myCPU/ 目录中。
- 2) 修改 func 配置文件——mycpu_env/func/include/test_config.h, 选择 exp9 的配置, 编译。(如果是通过压缩包 exp9.zip 获取实验开发环境的, 请跳过该步骤。)
- 3) 打开 gettrace 工程——mycpu_env/gettrace/gettrace.xpr。(该 Vivado 工程中的 IP 核是使用 Vivado 2019.2 创建的, 如果使用更高版本的 Vivado 打开, 请参考附录 D.4 节进行 IP 核升级。)运行 gettrace 工程的仿真(进入仿真界面后, 直接点击 run all 等待仿真运行完成), 生成新的参考 trace 文件 golden_trace.txt (mycpu_env/gettrace/golden_trace.txt)。要等仿真运行完成, golden_trace.txt 才有完整的内容。(如果是通过压缩包 exp9.zip 获取实验开发环境的, 请跳过该步骤。)
- 4) 进入 mycpu_env/soc_verify/soc_bram/run_vivado/ 目录下启动验证 myCPU 的工程。如果该目录下尚未创建工程, 请参照附录 D.2 节介绍的步骤, 利用该目录下的 create_project.tcl 文件创建工程。如需要, 请参考附录 D.4 节进行 IP 核升级。如果该目录下已有前一实践任务创建过的工程, 可以在打开工程后, 参照附录 D.3 节介绍的步骤, 更新项目中 CPU 实现文件的列表。
- 5) 参考 4.4.5.2 节, 对工程中的 inst_ram 重新定制。(如果是通过压缩包 exp9.zip 获取实验开发环境的, 请跳过该步骤。)
- 6) 在验证 myCPU 的工程中运行仿真(进入仿真界面后, 直接点击 run all), 进行功能验证与调试, 直至仿真测试通过。
- 7) 在验证 myCPU 的工程中综合实现后生成二进制码流文件, 进行上板验证。(如无硬件实验平台, 请跳过该步骤。)